

RCAAS.tech

Developer Guide

How the 3D tour platform is built: the architecture, the data, every API route and every module, and how to run, test, change and deploy it.

Generated from the code on 2026-10-05, at commit `639a8ee` with `uncommitted changes`, by `npm run docs:dev-guide`. Regenerate it after changing the code; the API, modules, pages, settings and known gaps update themselves. For what the product does for clients, read **RCAAS-Feature-Book.pdf** beside this file. The living, always-current brief is `CLAUDE.md` at the repository root.

Contents

- | | | | |
|----------|--------------------------------------|-----------|----------------------------|
| 1 | Read this first | 10 | Pages |
| 2 | What the product does | 11 | The 3D tour engine |
| 3 | How it fits together | 12 | Module index |
| 4 | Running it on your computer | 13 | Testing |
| 5 | Where things are | 14 | Settings |
| 6 | Rules that override feature requests | 15 | Deploying |
| 7 | The data | 16 | Recipes for common changes |
| 8 | Accounts and permissions | 17 | Known gaps |
| 9 | API reference | | |

1 Read this first

RCAAS.tech turns LiDAR scans of real places (hotel lobbies, banquet halls, college labs, heritage courtyards) into 3D tours that people walk through in a web browser, and turns visitors into enquiries and booking requests. It is two programs in one repository:

- **The website and studio** (`src/`): a Next.js 16 app. Visitors see the marketing site, the tours (`/tour`, `/t/:project`) and clients' websites (`/s/:project`). The team authors tours in the studio (`/studio`).
- **The API** (`server/`): an Express server. It stores spaces, projects, websites, bookings and enquiries, serves the scan files, and sends emails and texts.

Three things make this codebase unusual, and most surprises come from them:

- **The 3D engine is a vendor SDK.** Splats are drawn by the XGRIDS LCC Web SDK (`src/vendor/`), which owns a single WebGL renderer per page. The only documentation is `src/vendor/README.md`. Never guess what it can do.
- **Data is JSON files** in a data folder (`DATA_DIR`), behind small store modules. Only studio accounts can live in Postgres so far. A store's functions are its contract, so a database can replace one store at a time.
- **Publishing is a snapshot.** Editing in the studio never changes a live tour. Publishing writes a numbered copy, and visitors only ever read those copies.

New here? Read chapters 3 to 8 in order, then keep chapters 9 and 12 open as a reference while you work.

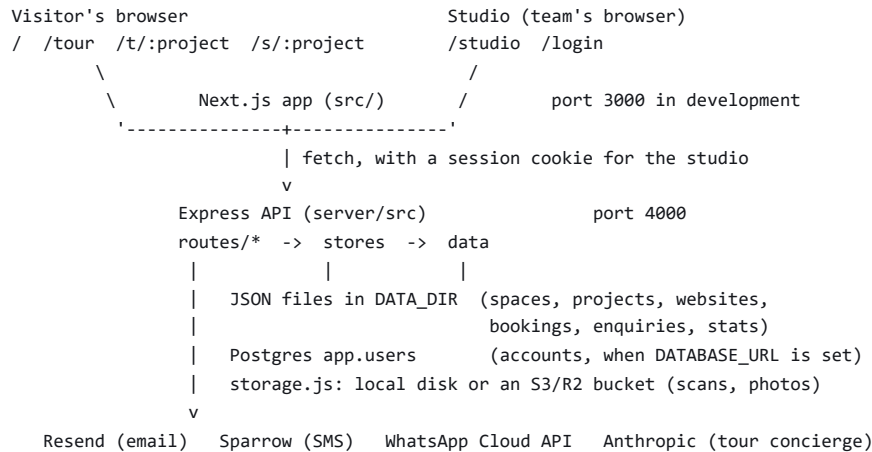
2 What the product does

A client (a hotel, restaurant, venue, college or heritage site) is a **project**. Each scanned room or area is a **space**. A space holds a scan, a start view, **hotspots** (information, media, links to other spaces, and Book now on tables, rooms and halls), **camera tracks** (authored flythroughs, stored in `tracks`), invisible **collision boxes**, and settings such as day and night versions.

Visitors walk, fly or orbit through a space, open hotspots, and send an **enquiry** or a **booking request** without leaving the tour. A project can also have a **website** at `/s/:project` (draft, preview, client review link, scheduled publish) with room, table and hall booking, menus, offers, reviews and an FAQ. The team answers everything in the **Reservations** inbox, and the monthly **report** shows visits, time spent and the path from visit to confirmed booking.

The feature-by-feature description, with screenshots, is **RCAAS-Feature-Book.pdf**.

3 How it fits together



Three requests, end to end

A visitor opens a tour

1. `/tour?space=:id` (or `/t/:project/:space`, which redirects there) renders `TourClient`, which reads the project's published spaces.
2. `GET /api/scenes/:id/published` returns the latest snapshot. Drafts are never served to visitors.
3. `useSceneManager` loads the scan through the LCC SDK from `/api/assets/:assetId/...` (byte ranges, cached for a year), or a `.glb` model, or a 360 video. `withColliders` adds collision boxes.
4. `useLccWalker` moves the camera (Walk, Fly, Orbit, Viewpoints) and keeps it out of walls. `HotspotMarkers` draws hotspots projected each frame.
5. The tour sends visit counts to `POST /api/stats` (totals only), and an enquiry goes to `POST /api/leads`, which emails the project's team.

The team edits and publishes a space

1. `/studio/:project` loads the draft with `GET /api/scenes/:id`. Access is checked by `sceneGuard` (`server/src/access.js`).
2. Edits change the in-memory scene document in `src/lib/sceneDoc.ts`. `history.ts` gives undo and redo, and the header shows unsaved changes.
3. Save sends the whole document with `PUT /api/scenes/:id`. The server keeps publish fields it owns and never lets a stale tab overwrite them.
4. Publish (`POST /api/scenes/:id/publish`) runs `publishChecks`, then writes `scenes/:id@n.json` once and never again. Revert, restore and version history work from those files.

A guest asks to book a table from a hotspot

1. The tour's `HotspotBookCard` loads free times from `GET /api/sites/:project/requests/tables`.
2. `POST /api/sites/:project/requests` checks the hotspot is published in that project, then checks again inside the project's write queue (`withReservations`) that the table is still free. A clash gets 409.
3. The request is stored in `reservations/:project.json`, and the team gets an email and text (`notify.js`, `mailer.js`).
4. The team confirms or declines in Reservations (`PATCH /api/sites/:project/reservations/:rid`), and the guest is told by text, WhatsApp or email.

4 Running it on your computer

```
# Node 24 (the tests run TypeScript directly), and Chrome for screenshots and this guide
npm install
cd server && npm install && cd ..
cp server/.env.example server/.env      # set SESSION_SECRET, EMBED_TOKEN_SECRET, EDITOR_PASSWORD
npm run dev                             # starts the API and the site, on free ports
```

Open the site, go to `/login?mode=signup` and create an account with the team access code (the `EDITOR_PASSWORD` you set). The first account ever created becomes the admin.

- **Scans are not in git.** `server/src/data/assets/` is ignored, so a new checkout has the spaces' JSON but no scans, and those tours show their start screen without 3D (the studio's publish panel warns). Copy the asset folders from another machine, or set `ASSET_DRIVER=s3` so every machine reads one bucket.
- **Accounts:** without `DATABASE_URL` they are JSON files in `DATA_DIR/users`. With it (Supabase or any Postgres), they live in `app.users`. `npm run db:import-users` (in `server/`) copies file accounts in.
- **Windows and Git Bash:** prefix variables that hold paths with `MSYS_NO_PATHCONV=1`. There is no Python on the main development PC, so scripts use Node.
- After changing the scene loaders (`useSceneManager`, loader options), hard-reload: the SDK renderer is a page singleton, which is also why React StrictMode is off.

Commands

Where	Command	Runs
root	<code>npm run dev</code>	<code>node scripts/dev.mjs</code>
root	<code>npm run dev:web</code>	<code>next dev --webpack</code>
root	<code>npm run build</code>	<code>next build --webpack</code>
root	<code>npm run start</code>	<code>next start</code>
root	<code>npm run lint</code>	<code>eslint .</code>
root	<code>npm run test</code>	<code>node --test test/*.test.mjs && cd server && npm test</code>
root	<code>npm run docs:dev-guide</code>	<code>node scripts/dev-guide.mjs</code>
server/	<code>npm run dev</code>	<code>node --watch src/index.js</code>
server/	<code>npm run start</code>	<code>node src/index.js</code>
server/	<code>npm run test</code>	<code>node --test</code>
server/	<code>npm run assets:push</code>	<code>node scripts/push-assets.js</code>
server/	<code>npm run make-admin</code>	<code>node scripts/make-admin.js</code>
server/	<code>npm run db:import-users</code>	<code>node scripts/import-users.js</code>
root	<code>npx tsc --noEmit</code>	Type-check the site
root	<code>node scripts/shot.mjs <url> <out.png></code>	Full-page screenshot through headless Chrome

5 Where things are

Folder	What is in it
src/app	Next.js routes. (site) is the marketing site, studio the authoring app, tour and t the tours, s client websites. Chapter 10 lists every page.
src/components	React components. App.tsx is the 3D canvas, EditorShell.tsx the whole studio UI (about 2,000 lines: search it, don't read it whole), Viewer.tsx the visitor's tour UI.
src/lib	Client logic without UI: the scene document, camera and collision maths, booking rules, API calls (api.ts), translations (i18n.ts).
src/@types	Shared TypeScript types: hotspots, scenes, viewpoints, uploads, the vendor SDK.
src/vendor	The XGRIDS LCC Web SDK and its README, the only documentation for it.
server/src	The API: index.js wires it up, routes/ are the endpoints, and the stores (store.js, usersStore*.js, reservations.js, leadsStore.js, stats.js, activity.js) own the data.
server/db	Postgres migrations, applied in name order when the API starts with DATABASE_URL set.
server/scripts	One-off tools: make an admin, push assets to the bucket, import accounts into Postgres.
server/test, test	The API tests (a real server on a free port) and the client tests (pure logic).
scripts	dev.mjs (start both programs), shot.mjs (screenshots), dev-guide.mjs (this document).
deploy	VPS deploy: deploy.sh, pm2 ecosystem.config.cjs, nginx.conf, and its README.md.
public	Static files shipped to every visitor. Never put masters here: they go in media-src/ (ignored by git).

6 Rules that override feature requests

These come from `CLAUDE.md` and win over any feature request. Each exists because breaking it caused, or would cause, a real problem.

- The scene JSON is renderer-agnostic. No SDK concepts in the schema. Assets are referenced by id, never by path or URL.
- A new scene-doc field must tolerate its absence (old docs): default it on load (`sceneDoc.ts`) or in `migrate.js` `fillLateDefaults`. Sanitize numbers that reach the camera.
- Publishing snapshots a version, and visitors read only snapshots. Studio edits never change a live tour.
- The enquiry path works even when WebGL/splat fails (`App.tsx` `no-WebGL2` branch).
- Audio never autoplays without a gesture.
- Three.js is pinned to r164 (the SDK's version). The SDK renderer is a page singleton, so there is no React StrictMode, and after editing `useSceneManager` /loader options you must hard-reload.
- Never guess an SDK capability. `src/vendor/README.md` is the only doc. `renderer.root` is undocumented but deliberately used (transform + collision move together).
- Never decimate a scan to hit a size: LCC streams by LOD. Budget first-frame bytes and resident memory instead.
- Production data lives in `DATA_DIR` outside the checkout. The studio writes scene JSON, and `server/src/data/scenes` in git is only seed content.
- Masters (raw video, FBX sources) go in `media-src/` (gitignored), never in `public/`. Everything in `public/` ships to every visitor.
- Ask before adding a dependency.

7 The data

The data folder

Everything except scans and accounts is JSON under `DATA_DIR` (default `server/src/data`; in production a folder outside the checkout). Each store module is the only code that reads or writes its folder.

Path	Owner	Holds
<code>scenes/:id.json</code>	<code>store.js</code>	A space's draft scene document (below).
<code>scenes/:id@n.json</code>	<code>store.js</code>	Published snapshot <i>n</i> . Written once (wx), never changed. Visitors read only these.
<code>properties/:id.json</code>	<code>store.js</code>	A project: title, ownerId, members, staff, brand theme and logo, contact info, enquiry email addresses.
<code>sites/:project.json</code>	<code>routes/sites.js</code>	<code>{ draft, published, publishedAt, scheduled?, review? }</code> . <code>cleanSite()</code> is the schema: it trims and defaults every field.
<code>reservations/:project.json</code>	<code>reservations.js</code>	Every booking and waitlist entry, written through one queue per project. kind: table, stay (room), event (hall), request (a hotspot's Book now, with of: table, room or hall), wait. Status: requested, confirmed, declined, cancelled; a wait is waiting, notified or removed.
<code>leads/</code>	<code>leadsStore.js</code>	Enquiries from tours and websites.
<code>stats/:project/:month.json</code>	<code>stats.js</code>	Visit totals per space and day, hotspot opens, booking intents. No cookies, IPs or anything personal.
<code>activity/</code>	<code>activity.js</code>	Who did what in each project (published, invited, confirmed).
<code>users/</code>	<code>usersStore-file.js</code>	Accounts when no <code>DATABASE_URL</code> is set, one file per account, named by a hash of the email.
<code>assets/:assetId/...</code>	<code>storage.js</code>	Scans, models, photos, audio, floor plans. Or the S3/R2 bucket with <code>ASSET_DRIVER=s3</code> . Always addressed by asset id, never by path.

The scene document

One JSON document per space, renderer-agnostic: nothing in it is specific to the SDK.

`server/src/migrate.js` upgrades old documents when they are read (current version 2), and

`src/lib/sceneDoc.ts` defaults anything missing when the studio loads one. A new field must survive being absent.

Field	Holds
<code>id, version, title, tagline</code>	Identity and the schema version.
<code>propertyId, ownerId</code>	The project it belongs to, and who created it (decides access while it is in no project).
<code>splat</code>	format (an LCC scan, a .glb model, or 360 video) and variants <code>.high medium low</code> , each <code>{ assetId, meta }</code> , where <code>meta</code> is the index file inside the asset.
<code>spawn</code>	The start view: position, yaw, eyeHeight.
<code>hotspots[]</code>	Each with <code>id</code> , type (text, image, video, audio, link, portal, table, room, hall), position, radius, label and a payload for its type (media, text, capacity, price, booking link).
<code>tracks[]</code>	Camera tracks (viewpoints): label, length in seconds, waypoints, thumbnail, optional narration and transcript.
<code>colliders[]</code>	Invisible collision boxes: position, size, turn.
<code>night, building, floor, neighbours</code>	The night version of a space, where it sits in a multi-building site, and linked spaces.
<code>status, publishedVersion, publishedAt, embed</code>	Owned by the publish and embed routes. A studio save can never change them.

Postgres

`server/src/db.js` holds the connection, a `tx()` helper and `migrate()`, which applies `server/db/*.sql` in name order on start and records them in `app.schema_migrations`. Tables live in the `app` schema, which Supabase does not expose through its public API, with row-level security on. Nothing is Supabase-specific: moving providers is `pg_dump` plus a new `DATABASE_URL`.

8 Accounts and permissions

- **Sessions:** a signed JWT in the httpOnly cookie `splatspace_session` (12 hours, SameSite=Lax, Secure in production), made by `middleware/auth.js`. A password change or reset ends every older session, because `staleReason` compares the token's issue time with the account's `passwordChangedAt`.
- **Accounts** (`usersStore.js` picks the file or Postgres version): script password hashes with a random salt. Signing up needs the team access code. Password reset is a one-time link (24 hours) made by an admin or emailed by Forgot password.
- **Roles:** **admin** manages team accounts (the Team panel), **editor** is a team member, and **staff** is a client's own staff, who can only answer their project's enquiries and bookings. Roles are read fresh from the account on every request, not from the cookie.
- **Projects:** visible to their owner and the members they add. Being admin does not show other people's projects. Spaces and assets are judged by their project (`access.js`: `sceneGuard`, `assetGuard`, `refuseStaff`).
- **Public endpoints** (enquiries, bookings, stats, auth) are rate-limited per connection. Forms carry a hidden field and their render time, to turn away bots.
- **Embeds:** each space has an embed key and an optional list of allowed sites, checked when `/tour` is framed by another site.

9 API reference

92 routes, read from `server/src/routes` when this guide was built. **Access:** *Public* needs nothing; *Signed in* needs a studio session; *with access to it* also checks the space or asset belongs to a project you can see; *Admin* needs the admin role. Routes marked Signed in may check further inside (a project's owner, a staff account's project). The description is the comment above the route in the code. Bodies and replies are JSON unless noted; errors are `{ error }` with a message meant for people.

`routes/assets.js`

	Path	Access	What it does
POST	<code>/api/assets</code>	Signed in	Start an upload: a new asset id to send a scan's files to (team accounts, not staff).
POST	<code>/api/assets/:assetId/files</code>	Signed in	One file of an upload, at its path inside the asset (chunked and resumable).
POST	<code>/api/assets/:assetId/finalize</code>	Signed in, with access to it	Finish an upload: check every file arrived, then move it into storage (?kind= for audio and models).
GET	<code>/api/assets/:assetId/*</code>	Public	Serve a stored file (scan tiles, photos, audio), with byte ranges and a one-year cache.
POST	<code>/api/assets/:assetId/floorplan</code>	Signed in, with access to it	(Re)build a space's floor plan, e.g. for a scan uploaded before plans existed.
PATCH	<code>/api/assets/:assetId/floorplan/rooms</code>	Signed in, with access to it	Name the rooms of a drawn plan: { names: { r1: 'Lobby', r2: " " } }. An empty name puts a room back to its number. Names survive a redraw.
DELETE	<code>/api/assets/:assetId/floorplan/upload</code>	Signed in, with access to it	Remove the floor plan image someone uploaded; the one drawn from the scan stays.
DELETE	<code>/api/assets/:assetId</code>	Signed in, with access to it	Delete an asset and all its files.

`routes/auth.js`

	Path	Access	What it does
POST	<code>/api/auth/signup</code>	Public	Create a studio account with the team access code. The first account ever becomes admin.
POST	<code>/api/auth/login</code>	Public	Sign in with email and password, or with only a password, the old shared team sign-in.
POST	<code>/api/auth/logout</code>	Public	Sign out: clear the session cookie.
POST	<code>/api/auth/reset</code>	Public	Set a new password from an admin's one-time reset link (<code>routes/team.js</code>), and sign in with it. Every older session of that account stops working.
POST	<code>/api/auth/password</code>	Public	Change your own password (the account drawer's Settings). Needs the current one. This browser gets a fresh session; every other session of the account stops working, and so does any reset link still waiting.
POST	<code>/api/auth/forgot</code>	Public	"Forgot password": emails the account a one-time link (the same kind an admin makes, <code>/login?reset=...</code> , a day, once). The answer is the same whether or not the email has an account, and the mail is sent after replying, so neither the reply nor its timing tells...
GET	<code>/api/auth/session</code>	Public	Who is signed in, with their current role (read from the account, not the cookie).

routes/concierge.js

	Path	Access	What it does
GET	/api/concierge/:project	Public	Whether the AI concierge is switched on for this project.
POST	/api/concierge/:project	Public	Ask the concierge; it answers only from what the project published (rate-limited).

routes/embed.js

	Path	Access	What it does
GET	/api/embed/check	Public	May this page embed the space's tour? Checks the embed key and the allowed sites.
GET	/api/embed/:id	Signed in, with access to it	A space's embed settings and its current key.
POST	/api/embed/:id	Signed in, with access to it	Change embedding: `rotate` retires every old embed code, `sites` sets up to 20 allowed sites.

routes/leads.js

	Path	Access	What it does
POST	/api/leads	Public	An enquiry from a tour or website: filed with its project and emailed to its team (rate-limited).
GET	/api/leads	Admin	Every project's enquiries: admins only. A project's own are at /api/properties/:id/leads, for anyone who can see that project.

routes/properties.js

	Path	Access	What it does
GET	/api/properties/:id/theme	Public	Public: a project's branding and WhatsApp number, for its tours. Nothing else about it.
GET	/api/properties/:id/leads	Signed in	A project's enquiries, and the addresses their emails go to.
GET	/api/properties/:id/report	Signed in	The monthly client report (studio/report/[property]): visits and time per space from stats.js, enquiries per space from the leads; the path to a booking (visits, hotspots opened, cards opened, enquiries, booking requests, confirmed) and the hotspots that got...
GET	/api/properties/:id/leads.csv	Signed in	The same, for Excel.
PUT	/api/properties/:id/lead-emails	Signed in	Owner or admin: who new enquiries are emailed to.
PUT	/api/properties/:id/theme	Signed in	Brand name, accent colour, heading font.
PUT	/api/properties/:id/logo	Signed in	Upload the project's logo (PNG, JPEG or WebP, as the raw body).
DELETE	/api/properties/:id/logo	Signed in	Remove the project's logo.
PUT	/api/properties/:id/info	Signed in	What the project says about its brand (store.js setPropertyInfo). Owner only.
GET	/api/properties	Signed in	Every property this session can see, with how many spaces it holds. Projects from before accounts (no owner) come too, but only as a name to claim.
POST	/api/properties/:id/claim	Signed in	Make a project from before accounts yours: from then on only you (and whoever you share it with) can see it.

	Path	Access	What it does
GET	/api/properties/:id	Signed in	One project with its members' names; a staff account gets only what staff may see.
POST	/api/properties	Signed in	Create a project (team accounts only).
GET	/api/properties/:id/activity	Signed in	Who did what in this project, newest first. Anyone who can see it.
PATCH	/api/properties/:id	Signed in	Title only — the id never changes (see renameProperty). Owner or admin.
DELETE	/api/properties/:id	Signed in	Its spaces are kept and become unfiled; nothing published changes. Owner or admin.
POST	/api/properties/:id/members	Signed in	Share the project with a teammate, by email: as a member (full access), or with `as: 'staff'` as the client's staff. Owner only.
DELETE	/api/properties/:id/members/:userId	Signed in	Take someone off a project (its owner only).

routes/reservations.js

	Path	Access	What it does
GET	/api/sites/:id/booking	Public	Every tour asks whether its project takes table bookings, so "no" is an ordinary answer here, not a 404 that each visitor's browser logs as an error. With it, `places`: every dining place taking bookings (the main one first), for the tour's choice.
GET	/api/sites/:id/availability	Public	Free table times on a day, at the main or another dining place (? outlet=).
GET	/api/sites/:id/preview/availability	Review link key, or signed in	Studio preview: the draft's tables and hours, against the real bookings. Never books.
GET	/api/sites/:id/stays	Public	Same as /booking: every tour asks, and "no rooms online" is an ordinary answer.
GET	/api/sites/:id/stays/availability	Public	Rooms left per night: the room booking calendar.
GET	/api/sites/:id/preview/stays/availability	Review link key, or signed in	Studio preview: the draft's rooms, against the real bookings. Never books.
GET	/api/sites/:id/events	Public	Same as /booking and /stays: every tour asks, and "no events online" is an ordinary answer.
GET	/api/sites/:id/events/availability	Public	Which halls are taken, and for which part of each day.
POST	/api/sites/:id/reservations	Public	A guest's table booking request from the website.
POST	/api/sites/:id/stays	Public	A guest's room booking request from the website.
POST	/api/sites/:id/events	Public	A guest's hall (event) booking request from the website.
GET	/api/sites/:id/requests/tables	Public	When each table in a space is free on a day (no names, nothing personal): the tour's Book now for tables. `tables`: the ones it knows; a table not among them (a draft's) has only the hours to go on.
GET	/api/sites/:id/requests/halls	Public	Which parts of a day a hall hotspot is already asked for (nothing personal): the tour's Book now strikes them out, the way the website's hall booking does.
POST	/api/sites/:id/requests	Public	Book now on a table, room or hall hotspot in the tour. Tables and halls refuse a clash (409).
POST	/api/sites/:id/waitlist	Public	Join the waitlist for a full day; the guest is told when a place frees up.
GET	/api/sites/:id/reservations	Signed in	The Reservations inbox: every booking and waitlist entry (the team, or the project's staff).

	Path	Access	What it does
PATCH	/api/sites/:id/reservations/:rid	Signed in	Confirm, decline, cancel or reopen a booking, re-checked so it can't double-book; tells the guest.

routes/scenes.js

	Path	Access	What it does
GET	/api/gallery	Public	Public: every published space, for the gallery page. ?property=<id>: just that project's (its hub page, /t/<id>).
GET	/api/gallery/:id/thumb.jpg	Public	Public: a published space's picture (its first view's thumbnail, kept as a data URL in the doc) as a real image, for the gallery and link previews.
GET	/api/scenes/:id/published	Public	Public: what visitors see. Never the draft (\$7.5, constraint 6). Its view thumbnails go as links (below), not the data URLs the doc keeps: ten of them are ~700 KB the tour would fetch before its first frame.
GET	/api/scenes/:id/published/thumbs/:track.jpg	Public	A published camera track's thumbnail, as a JPEG.
GET	/api/scenes/:id/publish	Signed in, with access to it	Studio: publish state + what would block or warn, without publishing.
POST	/api/scenes/:id/publish	Signed in, with access to it	Publish: run the checks, write snapshot n, and return warnings (missing scan files among them).
POST	/api/scenes/:id/unpublish	Signed in, with access to it	Take the space offline; its snapshots stay as history.
POST	/api/scenes/:id/revert	Signed in, with access to it	Throw away draft changes: copy the published version back into the draft.
GET	/api/scenes/:id/versions	Signed in, with access to it	Studio: every published version, newest first.
POST	/api/scenes/:id/restore	Signed in, with access to it	Studio: bring back published version N as the draft; with `publish: true`, put it live at once (as a new version, so the history keeps every step).
POST	/api/scenes/:id/property	Signed in, with access to it	Studio: move a space into a property, or out of all of them with null.
GET	/api/scenes	Signed in	The studio's space list: only the spaces your projects allow. (Visitors never needed it: the tour and gallery read published copies. It was public until 2026-09-27, which listed everyone's drafts to anyone.)
GET	/api/scenes/:id	Signed in, with access to it	Full draft document — splat pointer, spawn, hotspots, tracks, theme. The studio's, so only for those who may work on the space (visitors: /published).
PUT	/api/scenes/:id	Signed in, with access to it	Editor save. Session-auth only for now — add embed-token write scoping if the editor ever needs to save from an embedded context (it doesn't today).

routes/sites.js

	Path	Access	What it does
GET	/api/sites	Public	Every published website, for the sitemap: which projects, and when they last changed. Only what /s/<project> already shows anyone.
GET	/api/sites/:id	Public	A project's published website, ready to render at /s/:project.
GET	/api/sites/:id/preview	Signed in	Studio: the saved draft, shaped exactly like the public site, to see it before publishing.

	Path	Access	What it does
GET	/api/sites/:id/draft	Signed in	The website draft for the editor, with the project's spaces and viewpoints for its pickers.
PUT	/api/sites/:id/draft	Signed in	Save the website draft (cleaned and capped by cleanSite).
POST	/api/sites/:id/publish	Signed in	Publish the saved draft as the live website.
POST	/api/sites/:id/schedule	Signed in	Scheduled publishing (2026-09-28), e.g. a new menu at midnight: the draft as it is when scheduled goes live at `at` (publishDue, checked every SCHEDULE_TICK_MS by index.js). Edits after scheduling wait for the next publish;
DELETE	/api/sites/:id/schedule	Signed in	Cancel a scheduled publish.
POST	/api/sites/:id/review	Signed in	Make, or keep, the client review link for the draft.
DELETE	/api/sites/:id/review	Signed in	Turn the client review link off.
GET	/api/sites/:id/review/feedback	Signed in	The client's comments and approval, for the team.
PATCH	/api/sites/:id/review/comments/:cid	Signed in	Mark a client's comment resolved ('resolved: true') or open again.
GET	/api/sites/:id/review	Public	The draft website for a client's review link (?key=), no account needed.
POST	/api/sites/:id/review/comments	Public	A client's comment on the draft, pinned to a section (review link key).
POST	/api/sites/:id/review/approve	Public	The client approves the draft as it is now (review link key).
POST	/api/sites/:id/images	Signed in	Upload a website photo (PNG, JPEG or WebP, up to 25 MB, raw body); its size goes in its name.

routes/stats.js

	Path	Access	What it does
POST	/api/stats	Public	Tour beacon: counts visits, time per space, hotspot opens and booking intents. Totals only; always 204.

routes/team.js

	Path	Access	What it does
GET	/api/team	Admin	Every account, for the admin's Team panel.
PATCH	/api/team/:id/role	Admin	Make an account admin or editor, never leaving the team with no admin.
POST	/api/team/:id/reset	Admin	A one-time link, good for 24 hours, to send to the person.
DELETE	/api/team/:id	Admin	Remove an account (not your own, never the last admin); its projects move to you.

Health check: GET /api/health answers { ok }, and 503 when the database is set but down.

10 Pages

URL	File	What it is
/about	(site)/about/page.tsx	About, as a page of a surveyor's field book: the register of sites we've measured, the mission on a contour plate (or over the team's film, when public/media/about.* is delivered), and the kit. Styles: landing.css "Field book".
/contact	(site)/contact/page.tsx	Contact, as a page of a surveyor's field book: the sheet a crew fills in before a site visit, and the office marked on a contour plate. Styles: landing.css "Field book".
/cookies	(site)/cookies/page.tsx	The Cookie Policy: the one sign-in cookie and what stays in the browser (PolicyPage).
/gallery	(site)/gallery/page.tsx	Every published space. Rendered per request so a publish shows up at once.
/how-it-works	(site)/how-it-works/page.tsx	How it works: capture, process, author, publish, and the FAQ, marked up as FAQPage for search and AI answers.
/	(site)/page.tsx	The home page: the hero film, why a walkable tour sells, how visitors move, the specs, the steps and the sectors. Its words live in lib/siteContent.ts.
/privacy	(site)/privacy/page.tsx	The Privacy Policy, written to match what the code collects and sends (PolicyPage).
/security	(site)/security/page.tsx	The Security Policy: how accounts and data are protected, and how to report a problem (PolicyPage).
/services	(site)/services/page.tsx	Services: everything one scan can become (lib/siteContent.ts SERVICES).
/terms	(site)/terms/page.tsx	The Terms of Service (PolicyPage).
/work	(site)/work/page.tsx	Work: delivered and live projects, from lib/siteContent.ts WORK.
/llms.txt	llms.txt/route.ts	/llms.txt (llmstxt.org): the site in plain Markdown for AI assistants and answer engines, built from the same content as the pages so it never drifts.
/login	login/page.tsx	Fully static now — AuthPanel reads ?mode=?next= itself via useSearchParams(), which is what lets this page prerender at build time instead of requiring the request's query string up front. useSearchParams() requires a Suspense boundary; the fallback only shows for the instant before hydration reads the real URL.
/robots.txt	robots.txt/route.ts	Every crawler, search engines and AI assistants alike, may read the public pages. Private pages carry a noindex tag instead of a Disallow here, since a crawler barred from a page can't see that tag. A plain route, not app/robots.ts: Next's metadata-route loader writes the file's path into a JS string, and the apostrophe in this checkout's path ("Prabhat's projects") breaks it.
/s/:property	s/[property]/page.tsx	A project's website (/s/<project>): the editorial page the studio writes at /studio/<project>/site, around the project's live tour. The tour is the published one, embedded unchanged; "View in 3D" on a room sends it there. Server-rendered, so every word is in the HTML and indexable. Rendered per request, so a publish shows at once.
/s/:property/review	s/[property]/review/page.tsx	The client's review of a website draft (/s/<project>/review?key=...), from the link the studio shares: the draft as visitors will see it, with comment pins. "Comment" then a click anywhere pins a note to that spot (its section, and where in it, so pins stay put as the page reflows); "Approve this draft" signs it off. No account: the link's key is the pass. Bookings and enquiries don't send from here.

URL	File	What it is
/sitemap.xml	sitemap.xml/route.ts	The marketing pages, every published client website (/s/), and every project's tour page (/t/). Single tours redirect to /tour, so they're left out. Rebuilt hourly, so a new publish shows up without a deploy. A plain route rather than app/sitemap.ts for the reason in robots.txt/route.ts.
/studio/:property	studio/[property]/page.tsx	The editor, inside one client's project (a "property" in code, \$0.2). The space list is scoped BEFORE the 3D mounts, so the renderer never starts streaming another client's model. A project with no spaces yet goes straight to the uploader.
/studio/:property/plan/:space	studio/[property]/plan/[space]/page.tsx	A space's floor plan as a client deliverable: a title sheet with the project's logo, the plan the studio was showing (?img=floorplan/...), and a room schedule when the plan was drawn from the scan. "Save as PDF" is the browser's print dialog, like the monthly report.
/studio/:property/report	studio/[property]/report/page.tsx	The path to a booking: from a visit to a confirmed booking, each step as a bar against the visits, with how many went on from the step before. The first three steps are counted in the tour; the rest are what was saved, so they include enquiries and bookings made on the website too.
/studio/:property/reservations	studio/[property]/reservations/page.tsx	Tables, rooms or events: the inbox's filter.
/studio/:property/site	studio/[property]/site/page.tsx	The website's looks (website.css [data-style]): key, name, what it is, paper, ink.
/studio/:property/site/preview	studio/[property]/site/preview/page.tsx	The website editor's Preview (/studio/<project>/site/preview): the saved draft, drawn by the same SiteView as the public page, so what you see is what visitors will get after Publish. Enquiries and bookings wait for it.
/studio	studio/page.tsx	Only its owner can rename, delete, rebrand or share it (an admin too sees only their own projects). The shared team password, with no account, can.
/t/:property/:space	t/[property]/[space]/page.tsx	/t/<project>/<space>: the space's tour. The tour itself still lives at /tour (it checks the space is published and belongs to a project before any 3D loads); this is the readable address the hub page links to. Anything else on the link (?lang=, ?embed=1) goes along with it.
/t/:property	t/[property]/page.tsx	A project's own page (2026-09-25): /t/<project>. The page a hotel links from its website — its name, logo and colour, every published space as a card that walks into the tour, and the enquiry form. Server-rendered, so it's fast and indexable, and it works with no 3D at all (the tour only loads when a visitor picks a space). Rendered per request so a publish or a branding change shows at once.
/tour	tour/page.tsx	The tour runs in the browser (TourClient). This server half only names the space for link previews: `/tour?space=<id>` pasted into WhatsApp shows the space, the hotel and a picture. Published spaces only, like the tour.

11 The 3D tour engine

- **The renderer** is the LCC SDK's, one per page. `App.tsx` mounts a react-three-fiber canvas (Three.js pinned to r164, the SDK's version) and wires the scene manager, walker, camera director, gizmo, collision boxes and the studio's `EditorApi`. Objects are built imperatively in `useEffect`, not with R3F JSX.
- **Loading:** `useSceneManager.ts` loads one space at a time: an LCC scan through the SDK, a `.glb` through `meshModel.ts` (with a BVH for collision), or 360 video through `panoModel.ts`. `renderer.root` is undocumented but used on purpose: moving it moves the model and its collision together.
- **Quality:** `deviceTier.ts` guesses a tier, then the tour measures frames per second and reloads a lower tier if needed, keeping the visitor's place. LCC streams by level of detail, so scans are never decimated to hit a size.
- **Moving:** `useLccWalker.ts` runs Walk, Fly, Orbit and Viewpoints, with capsule collision (`renderer.intersectsCapsule`) and floor probes from `collision.ts`. `useCameraDirector.ts` plays camera tracks and glides.
- **Hotspots:** projected to screen each frame into `hotspotProjector.ts`, and drawn as DOM by `HotspotMarkers.tsx` on its own animation frame, so React never re-renders at 60 fps.
- **When WebGL is missing** (old phones, some bots), `App.tsx` keeps the enquiry path working anyway.
- **Testing it:** headless Chrome renders splats in software at about 1.6 fps, so judge motion on a real GPU. In development, `window.__camera` exposes the camera for checking positions.

12 Module index

Every source file and the first comment in it, read when this guide was built. A file without a comment is a hint that one is owed.

Client logic: `src/lib`

File	Lines	What it is for
<code>src/lib/api.ts</code>	562	Client for the Node API in <code>/server</code> — the CLAUDE.md-planned Django + DRF backend, not started yet, stood up here in Node instead. Every call is best-effort from the app's point of view: the viewer must keep working off the scenes baked into <code>lib/scenes.js</code> if the API is unreachable, because "time to first frame under 5s" (CLAUDE.md) can't wait on a second server.
<code>src/lib/audio.ts</code>	133	Tour audio (CLAUDE.md §6.3). One <code>AudioContext</code> for the page, Web Audio API only, no library. Today there is one layer — hotspot clips — behind one gain node that is also the mute switch. Ambient and track music/narration join as their own gain nodes when they are built.
<code>src/lib/booking.ts</code>	93	Book now (CLAUDE.md §7.6) — the dates-and-guests half. Pure, no imports, so it is tested directly (<code>test/client.test.mjs</code>).
<code>src/lib/brandColor.ts</code>	99	Brand colour helpers (2026-09-28): read a logo's colours so the brand colour can be suggested from it, and pick readable text for any colour a button is filled with.
<code>src/lib/collision.ts</code>	289	Finding solid ground in an LCC scan.
<code>src/lib/deviceTier.ts</code>	197	Three quality tiers — high / medium / low (CLAUDE.md §8). Resolved once per session, before the SDK loads, because the tier decides which splat variant gets fetched.
<code>src/lib/editorActive.ts</code>	11	Editor vs. viewer, decided by route: <code>/studio</code> is the editor, everything else (today: <code>/tour</code>) is the visitor viewer. <code>`?view`</code> still forces the viewer, and from inside the studio ► Preview renders the real viewer anyway.
<code>src/lib/floorMap.ts</code>	56	The tour's floor map: the plan the server drew from the scan (<code>server/src/floorplan.js</code> → <code><asset>/floorplan/plan.json</code>), and the bridge between it and the 3D. Inside the Canvas, App writes where the camera is in the scan's own coordinates every frame (<code>mapPose</code>) and registers how to jump somewhere (<code>setMapGoto</code>); the map outside the Canvas reads and calls them.
<code>src/lib/history.ts</code>	88	Step-by-step undo and redo in the scene editor (2026-09-28), next to the version history's save points. After each change settles (0.4 s: a whole gizmo drag is one step), a snapshot of what the studio edits in the open space — hotspots, collision boxes, camera tracks, the model's placement, the start view, Book now — goes on its undo stack. Undo puts the one before back into the stores;
<code>src/lib/hotspotLayout.ts</code>	71	Where a hotspot's label card sits on screen (the callout look: a ring on the spot, a thin leader line, a card off to one side). Pure maths, no DOM, so it is tested directly — see <code>test/hotspotLayout.test.mjs</code> .
<code>src/lib/hotspotProjector.ts</code>	12	Bridge between the R3F frame loop and the DOM hotspot markers. Stage projects the active scene's hotspots to screen pixels every frame and writes them here; the DOM <code><HotspotMarkers></code> reads on its own rAF. Plain mutable object — no React state at 60fps.
<code>src/lib/i18n.ts</code>	513	The visitor tour's own words in English, Nepali or Chinese. Keys are the English text itself, so an untranslated string simply stays English and <code>`t()``</code> can wrap any sentence, wherever it comes from. <code>`{name}`</code> placeholders are filled from <code>`vars`</code> . What the hotel typed (space and hotspot names) is shown as written.
<code>src/lib/lccConfig.ts</code>	129	LCC load configuration.
<code>src/lib/media.ts</code>	36	Marketing footage, by slot name, from <code>public/media/</code> (list: <code>public/media/README.md</code>). Server-only (reads the disk): a slot shows only if its file is there, so a page never points at footage that hasn't been delivered yet; without it the page keeps its plain design. Pages are prerendered, so new files need a rebuild (or a dev server) to appear.

File	Lines	What it is for
src/lib/meshModel.ts	219	3D-model spaces: one .glb, made from an FBX/OBJ/PLY/gITF on upload by modelConvert.ts (upright, in metres, meshopt-compressed).
src/lib/mobileInput.ts	34	Shared mobile-input state.
src/lib/modelConvert.ts	288	3D models on upload: FBX, OBJ (+MTL), PLY, gITF/GLB -> one compressed .glb.
src/lib/modelPrep.ts	104	Pure geometry for preparing a 3D model on upload (modelConvert.ts): which way is up, and a floor to walk on for a point cloud. No three.js, no DOM, so `npm test` runs it directly.
src/lib/navMode.ts	51	Visitor navigation mode (CLAUDE.md §6.1): viewpoints (default — free look, tap the dock to dolly between authored stops) vs walk (explicit opt-in — WASD / joystick free-roam with collision). A space must always OPEN in viewpoints; walk must never become the default by accident.
src/lib/panoModel.ts	108	A 360 camera video as a space: the equirectangular video plays on the inside of a sphere that stays centred on the camera, like a sky. Looking round works in every mode; nothing walks off-centre and stretches the picture (useSceneManager also holds walking speed at 0 for these).
src/lib/sceneDoc.ts	400	Scene document — the renderer-agnostic layer (CLAUDE.md: "the schema is the actual IP"). Holds hotspots; camera tracks live in viewpoints.ts and spawn overrides in scenes.ts.
src/lib/scenes.ts	266	Hotel scenes.
src/lib/shareMeta.ts	27	The public address of this site, for canonical links, the sitemap and structured data. Set NEXT_PUBLIC_SITE_URL in production (.env.production); if it's forgotten, a production build falls back to NEXT_PUBLIC_API_URL, which is the public domain there (nginx serves both), never localhost.
src/lib/siteContent.ts	168	Copy for the marketing pages (/ , /work, /services, /how-it-works, /about). Facts come from GeoNova's published case studies (geonova.com.np). Keep them sourced: a number on these pages is a promise to a client. Scanner figures (± 1.2 cm, 200,000 points a second, 360°) are XGRIDS' published Lixel K1 specs (xgrids.com/intl/lixelk1).
src/lib/siteTemplates.ts	81	Starter projects (2026-09-28): what kind of place a new project is, and the website draft it starts with. Only what's safe to leave as is: sections that stay hidden until they're filled in (a menu with no dishes), booking set up but switched off (rooms with no photos or prices yet), and headings that read right for any place of that kind. Nothing here is live until the website is published.
src/lib/stats.ts	65	Visit and time counts for the monthly client report (server/src/stats.js). sendBeacon with text/plain: it survives the tab closing and needs no preflight.
src/lib/theme.ts	22	The studio and site theme (dark or light), kept in localStorage and set on <html data-theme>; layout.tsx applies it before the first paint.
src/lib/transform.ts	94	Scene transform (CLAUDE.md §7.2) — where the author placed the model.
src/lib/uiConfig.ts	67	Visitor-facing presentation settings, driven by the editor's Customize tab. Shared mutable object + subscription, read by the visitor components through `useUiConfig()`.
src/lib/upload.ts	282	Chunked resumable upload engine for the studio's Uploader (CLAUDE.md §7.1). Two independent jobs:
src/lib/useCameraDirector.ts	180	The cinematic engine behind the viewport buttons.
src/lib/useLccWalker.ts	537	True when a keystroke belongs to a text field rather than the camera.
src/lib/useSceneManager.ts	313	The SDK's per-scene renderer handle (collision, raycast, bounds — untyped, see @types/vendor.d.ts) plus our own load bookkeeping.
src/lib/useStudioSession.ts	27	Studio pages are for signed-in team members. Signed-out visitors go to /login and come back to `next`. Resolves true once it's fine to render. If the API can't be reached at all it lets the page through — the page then shows its own "server isn't answering" state, and every write is refused server-side anyway.
src/lib/viewpoints.ts	264	Viewpoints — the clickable "camera shots" for each scene.

File	Lines	What it is for
src/lib/walkerConfig.ts	60	Live, tunable walker + camera settings — shared mutable state, same pattern as mobileInput.js. The scene manager fills in scale-derived defaults on load; the editor panel's sliders write here; the walker and director read it every frame. Never put this in React state (it changes at slider-drag rates).

Components: `src/components`

File	Lines	What it is for
src/components/App.tsx	539	=====
src/components/AuthPanel.tsx	237	Only same-site paths — never an absolute or protocol-relative URL, which would turn ?next= into an open redirect.
src/components/BookingCard.tsx	97	The Book now card (§7.6): the room, the dates and party size if the hotel wants them, the price the hotel typed, and one button that continues on the hotel's own booking page with those choices filled in. We never take a booking or a payment here.
src/components/BrandInfoFields.tsx	46	The brand information fields (server/src/store.js setPropertyInfo), for the New project dialog and the project's Branding. Its website shows them.
src/components/ColliderBoxes.tsx	94	Studio only — App.tsx never mounts this in the tour or in Preview, so visitors never see a box, they only bump into it (collision.ts withColliders). Each box is drawn in its colour, its outline showing through the scan so a box behind a wall can still be found, and the selected one gets move arrows.
src/components/ConciergePanel.tsx	84	The AI concierge (server/src/routes/concierge.js), over the tour: a visitor's questions about the place, answered from what the project has published, in their language. When an answer is about a place in the tour, the tour goes there ('onShow'), and the answer offers to go again. The conversation lasts as long as the panel's page.
src/components/ContactSheet.tsx	63	The Contact page's field sheet: what a crew needs to know before a site visit, on ruled lines. "Write the email" opens the visitor's own email app with the answers filled in: no form backend, nothing stored here.
src/components/ContourPlate.tsx	45	Topographic contour lines around a survey benchmark: the line work of the Contact and About pages (landing.css .lp-contour). Drawn here, at build time, from a fixed seed, so it's the same plate on every visit and no script runs for it. The lines draw themselves in once (CSS), unless motion is reduced.
src/components/EditorShell.tsx	2067	=====
src/components/EnquiryPanel.tsx	188	The persistent CTA + enquiry panel (CLAUDE.md §6.4) — this is the product, not an add-on. Mounted independent of whether the 3D ever loads (constraint 5: "the CTA is never blocked by the 3D"), so it only needs a scene id and name, never 'state.ready'.
src/components/EventBooking.tsx	211	The whole day clashes with either half (server/src/events.js).
src/components/FloorMap.tsx	79	The tour's live floor map: rooms, walls and doors from the scan's own plan, a "you are here" dot with a view cone, and rooms you tap to go to. Shown only for spaces with two or more rooms: a single room doesn't need a map. The dot moves by writing attributes each frame, not React state.
src/components/Gizmo.tsx	155	Keeps the loaded model where the scene's transform says, in the studio and the tour alike, and rescales the walker when the author changes scale (§7.2: "re-derive, don't reload").
src/components/HotspotBookCard.tsx	319	Book now on a room, hall or table hotspot, over the tour (2026-09-30): what that kind of booking needs (the nights; the day and part of it; the day and time), a name and a phone, one button. It works with nothing else set up: the request lands in the studio's Reservations, where the team confirms or declines it, and what's booked is the hotspot itself (its name, price, deposit).
src/components/HotspotMarkers.tsx	282	How many hotspots show their card at once; the rest are rings until hovered.
src/components/JsonLd.tsx	59	schema.org structured data, so search engines and AI answers read who we are, what we do and our FAQ as facts, not guesses. '<' is escaped so the text can never close the script tag.

File	Lines	What it is for
src/components/NewProjectDialog.tsx	429	"Bar_Restro" (a folder), "Bar_Restro.zip", ".../Bar_Restro.lcc2" or "Lobby.fbx" -> "Bar Restro".
src/components/PasswordChange.tsx	45	"Change password" in the account drawer's Settings, in the site nav and the studio. `cls` is the drawer's class prefix (site.css / editor.css `-pw`).
src/components/PolicyPage.tsx	36	The layout of the policy pages (terms, privacy, security, cookies) and the contact block they share.
src/components/RoomBooking.tsx	408	The days of a month (YYYY-MM), and how many blank cells come first in a Monday-first week.
src/components/RoomCard.tsx	209	Book a room, over the tour: the dates, how many guests, the room (the one this space shows, when it's one the hotel lets online), a name and a phone, and one button. The same request as the website's room booking; all the rooms with their photos and the site plan are a link away (RoomBooking).
src/components/SiteGsap.tsx	122	What rises in as it scrolls into view, on every marketing page. Hidden before the first paint by landing.css (html.gs), so nothing flashes.
src/components/SiteMotion.tsx	277	The marketing pages' scroll-driven media, in the manner of XGRIDS' Lixel K1 page: a pinned scene played by the scroll (an image sequence, or the tour's photos flown through in CSS 3D), segmented tabs over one large picture, and a sticky media panel that changes with the step you're reading. Footage comes from src/lib/media.ts. Styles are in landing.css.
src/components/SiteNav.tsx	268	In page order: a link to the right of the current page slides the next page in from the right (nav-forward), one to the left from the left.
src/components/SitePage.tsx	97	The body of every marketing page under src/app/(site)/. The nav lives in that group's layout and stays put; this part slides: out one way and in from the other, by the direction the link was tagged with (SiteNav tags nav links by their order). Untagged navigations (browser back) just crossfade. The wrapper must be rendered by each page, not the layout: a layout persists, so it never enters or exits.
src/components/SplatField.tsx	201	The marketing illustration: a procedurally built lounge as a LiDAR point cloud, with a scan plane sweeping across it and turning every point it passes into a soft, coloured Gaussian splat. Plain 2D canvas — no WebGL, no SDK, nothing that competes with a real tour for GPU or bytes.
src/components/TableBooking.tsx	340	Where a table's chairs sit, in table-units from its centre, by its shape.
src/components/TableCard.tsx	203	Reserve a table, the way Book now books a room (\$7.6): one card over the tour — day, time, guests, the table (best available, or the one a table hotspot named), a name and a phone, and one button. The full floor plan is a link away for guests who want to pick by sight (TableBooking).
src/components/TeamDialog.tsx	102	The team list, opened from the studio's top bar. Admin-only — the server refuses the requests behind it for anyone else, and the button that opens this is itself hidden from non-admins (studio/page.tsx).
src/components/ThemeToggle.tsx	26	One switch for both surfaces. ` <html data-theme="">` is the source of truth — site.css and editor.css each flip their own tokens off it, so this button knows nothing about either. Stateless on purpose: the sun/moon swap is CSS (globals.css), so there is no state to hydrate and nothing to keep in step with the inline script in layout.tsx.</html>
src/components/TouchControls.tsx	99	Mobile controls. A floating thumb-stick in the bottom-left zone (it springs up wherever your thumb lands) plus Jump / Run. Look is the walker's own canvas drag listener — these overlays sit on top of the canvas so touches that start here never become a look-drag.
src/components/Uploader.tsx	374	What converting a 3D model is doing, before any bytes go up.
src/components/Viewer.tsx	902	The visitor experience. The room is the interface (\$10.2): the place name, a few icon buttons, the mode switch, the views bar and the enquiry button, and nothing else until the visitor acts. Reused verbatim as the studio's Preview (with `autoStart` + `tour`).
src/components/WaitlistForm.tsx	67	Join the waitlist (server/src/waitlist.js): shown by TableBooking and RoomBooking when what the guest wants is all taken. A name and a phone; if a table or room is let go, they're told by text (and email, if given) with a link back to book. Nothing is held for them.

API: server/src

File	Lines	What it is for
server/src/access.js	99	Who may work on what (2026-09-24 ownership, applied to spaces and assets 2026-09-25). A space belongs to its project ('propertyId'); you can work on it if you can see that project — its owner, a member, or an admin — or if it's in no project (or its project was deleted). An asset is judged by the spaces that use it; one no space uses yet (an upload in progress) is open to whoever is signed in, as it always was.
server/src/activity.js	63	Who did what, per project (2026-09-25): one append-only JSON-lines file per project under data/activity/ (gitignored: it carries names). Spaces in no project log to _unfiled. Writing an entry never fails the action it records — a full disk costs the log line, not the publish.
server/src/dataDir.js	12	Where every file-backed store keeps its data. Defaults to src/data; the integration tests point DATA_DIR at a temp folder so they never touch real scenes, users or leads. ASSET_DIR (storage.js) still overrides assets alone.
server/src/db.js	58	The Postgres connection (Supabase, or any Postgres: nothing here is Supabase-specific, so moving providers is pg_dump + a new DATABASE_URL). Only this server talks to the database; tables live in the 'app' schema, which Supabase's public Data API doesn't expose, with RLS on as well.
server/src/events.js	56	Event bookings (2026-09-29) for a hotel's or banquet venue's halls: the counterpart of stays.js's rooms and reservations.js's tables. The halls — how many each seats and holds standing, their size, photos and 3D space — are part of the site document (sites.js cleanEvents), so they go live on Publish. A guest asks for a hall on a day, for the daytime, the evening or the whole day;
server/src/floorplan.js	886	Floor plan from a Lixel Studio export, no Lixel Studio needed.
server/src/index.js	120	Listen on PORT. If it's taken by an earlier copy of this same API (VS Code closed without stopping it), stop that copy and take the port back; if something else has it, move up a port (at most ten) and say so, since the site must then be pointed at the new one (scripts/dev.mjs does that itself).
server/src/leadsStore.js	62	Lead storage (CLAUDE.md §6.4). One JSON file per lead under data/leads/<id>.json — same file-backed pattern as scenes (store.js) until Postgres exists. Leads are never edited after intake, only listed, so this is intentionally simpler than store.js (no update path).
server/src/mailer.js	64	Enquiry emails (2026-09-25), through Resend's HTTP API with Node's own fetch — no mail library. Off until RESEND_API_KEY is set in server/.env; without it enquiries are still stored and exported, just not emailed.
server/src/middleware/auth.js	90	Editor session auth. CLAUDE.md: "session auth for the editor, signed short-lived tokens for embeds" — this is the session half. A signed, httpOnly cookie carrying the account (routes/auth.js, usersStore.js), or no account for the legacy shared-password sign-in.
server/src/migrate.js	125	Scene schema migrations (CLAUDE.md §5.3). Run ON READ, not as a one-off script — a scene authored last month must open today without anyone remembering to run anything. store.js calls this on every getScene(); the migrated doc is only written back on the next save, never on read alone.
server/src/notify.js	84	Text messages to guests (2026-09-28): booking confirmations and waitlist news by SMS and/or WhatsApp, alongside the email mailer.js sends. Each is off until its keys are in server/.env; with neither, guests still get email if they left an address. Node's own fetch, no library.
server/src/reservations.js	120	Table bookings for a project's website (/s/<project>, #reserve). The restaurant's setup — its floor plan, tables, hours — is part of the site document (sites.js cleanBooking), so it goes live on Publish. Bookings are requests: the guest picks a table and a time, the restaurant confirms or declines from the studio.
server/src/routes/assets.js	672	Upload init/chunk/finalize + range-capable read (CLAUDE.md §7.1, §9).
server/src/routes/auth.js	193	Studio sign-up / sign-in.

File	Lines	What it is for
server/src/routes/concierge.js	175	The AI concierge in the tour (2026-09-28): a visitor asks a question in their own words and gets an answer from what the project has published — its brand information, its website (story, rooms, menu, booking hours) and its spaces (their hotspots and views) — and, where it helps, the tour goes to the place the answer is about. Claude through Anthropic's Messages API, Node's own fetch, no SDK.
server/src/routes/embed.js	94	Embeds (2026-09-25). A hotel pastes an iframe once, so a short-lived token can't be what lets it in. Instead each space has an embed key — an HMAC of its id and an embed `version`, no expiry — baked into the snippet the studio hands out. Retiring every old snippet is one bump of the version. Optionally a space lists the websites allowed to frame it;
server/src/routes/leads.js	115	Public lead intake (CLAUDE.md §6.4). Hostile-facing by definition — no auth, reachable from any embed — so: rate limit by IP, a honeypot field, a submission-time check, a size cap per field, and nothing stored or echoed back that could carry HTML.
server/src/routes/properties.js	353	Properties (CLAUDE.md §5.1) — one per client. Studio-only for now: the public hub page (/t/<property>) that would read these is not built yet, so there is no public read and no publish state here.
server/src/routes/reservations.js	620	Table and room bookings (reservations.js and stays.js have the rules). Public: what's free, and asking for a table or a room — no auth, so the enquiry form's defences: a rate limit, a honeypot, a fill-time check, capped fields, no HTML. Studio: the project's bookings of both kinds, and confirming or declining them (the guest is emailed either way, if they left an address).
server/src/routes/scenes.js	184	Spaces (scene documents): the studio's list, load and save, publishing with numbered snapshots, version history, and the public reads of published spaces and their pictures.
server/src/routes/sites.js	536	Project websites (/s/<project>): an editorial page around the project's live tour — its story, its rooms, photos, floor plan, a menu, and the enquiry form. The tour itself is the published one, embedded as-is. The studio edits a draft; publishing copies it to what the public page reads. Stored as data/sites/<project>.json: { draft, published, publishedAt, scheduled? }. Photos are the asset `site_<project>`.
server/src/routes/stats.js	56	Public visit beacons from the tour (stats.js). The tour sends them with navigator.sendBeacon as text/plain — a beacon can't make the preflight a JSON content type would need. Only published spaces count, and the project is looked up here, never taken from the beacon. Rate-limited per IP so a script can't inflate a client's numbers much.
server/src/routes/team.js	54	The team list — who has a studio account and their role. Admin-only: the console for promoting/demoting people, making a one-time password reset link (there's no email service yet: the admin sends it), and removing an account. A removed person is signed out everywhere at once (middleware/auth.js) and their projects pass to the admin who removed them.
server/src/stats.js	78	Visit counts for the monthly client report (2026-09-27). Totals only: no cookies, no IPs, nothing about a person — per project, per month, per space: how many visits, how many seconds, and visits per day. One small file per project-month under data/stats/ (gitignored). Writes to one file are queued, so beacons arriving together can't lose each other's counts.
server/src/stays.js	71	Room bookings for a project's website (/s/<project>, #stay), the rooms' counterpart of reservations.js's tables. The hotel's setup — its room types (or villas, one of a kind), how many of each, how many each sleeps, an optional site plan they sit on — is part of the site document (sites.js cleanStays), so it goes live on Publish.
server/src/storage-s3.js	178	The S3-compatible storage driver (2026-09-28): uploaded splats live in one cloud bucket instead of on the PC that uploaded them, so every PC — and the hosted API — serves the same files. Works with Cloudflare R2, Backblaze B2, AWS S3, MinIO, Wasabi, Supabase Storage: anything that speaks S3.
server/src/storage.js	148	Asset storage driver seam (CLAUDE.md §9). Routes and store.js never touch the filesystem for assets directly, only this module. Two drivers:

File	Lines	What it is for
server/src/store.js	575	File-backed scene store. CLAUDE.md's actual plan is Django + DRF + Postgres; this Node service is the pre-database stand-in, so the storage layer is isolated behind this one module — swapping in Postgres later means rewriting this file, not the routes that call it.
server/src/usersStore-file.js	210	Studio accounts. One JSON file per user under data/users/, named by a hash of the lowercased email — so an email never becomes a path, and lookup is one read. Same file-backed pattern as scenes/leads; usersStore-pg.js is the Postgres twin.
server/src/usersStore-pg.js	132	Studio accounts in Postgres (db/001_auth.sql). Same functions, results and errors as usersStore-file.js, which the routes can't tell apart. Password hashing is shared with it, so imported accounts keep their passwords.
server/src/usersStore.js	13	Studio accounts. With DATABASE_URL set (Supabase or any Postgres) they live in the database (usersStore-pg.js); without it, in JSON files under DATA_DIR/users (usersStore-file.js). Both export the same functions. Copy file accounts into the database with: npm run db:import-users
server/src/waitlist.js	35	The waitlist (2026-09-28): guests who found a day's tables, or a stay's rooms, all taken. They're kept with the bookings (reservations.js), marked kind: 'wait', and never hold a table or a room. When a booking is declined or cancelled, the ones who could now book are told (routes/reservations.js), first come first told; the studio can tell or remove one by hand.

Scripts

File	Lines	What it is for
scripts/dev-guide.mjs	464	Markdown's inline bits (bold, code) as HTML; enough for CLAUDE.md's bullets.
scripts/dev.mjs	145	npm run dev: the site (Next) and the API together, on free ports.
scripts/shot.mjs	56	not up yet
server/scripts/import-users.js	37	Copies the file accounts (DATA_DIR/users/*.json) into Postgres, keeping their ids and password hashes, so everyone signs in as before and every project's owner/member ids still match. Safe to run again: an email that's already in the database is left alone. The JSON files are not touched; unset DATABASE_URL to go back to them.
server/scripts/make-admin.js	45	Give an existing studio account the admin role, from the command line. The studio's Team panel can promote people, but only an admin can open it: accounts made before roles existed have none, so this is how the first admin is named. Run it where the API's data lives (this PC, or the host).
server/scripts/push-assets.js	53	Copy this PC's uploaded assets (server/src/data/assets, or ASSET_DIR) into the S3 bucket set in server/.env, so every PC and the hosted API can serve them. Run it once on each PC that has uploads, before or after switching ASSET_DRIVER=s3 on. Files already in the bucket at the same size are skipped, so running it again only sends what's new. Nothing is deleted, here or there.

13 Testing

- `npm test` runs the client tests (`test/client.test.mjs`: pure logic, TypeScript imported directly) and then the API tests twice, once storing files on disk and once on a fake S3 (`API_TEST_STORAGE=s3`).
- The API tests start a real server on a free port with a temporary `DATA_DIR`, and fake mail, SMS, WhatsApp and Anthropic servers that record what was sent. They set `DATABASE_URL` themselves, so they never touch the database in `server/.env`; set `API_TEST_DATABASE_URL` to run them against a throwaway Postgres.
- New pure logic gets one assert-style test in `test/client.test.mjs`. A new endpoint gets its behaviour, its refusals and its access checked in `server/test/api.test.js`.
- Every string shown in the tour needs Nepali and Chinese in `i18n.ts`; a test fails otherwise.
- CI (`.github/workflows/ci.yml`) runs lint, types, tests and a production build on every push and pull request.

14 Settings

API settings: `server/.env`

Setting	Example	What it does
PORT	4000	
CLIENT_ORIGIN	http://localhost:3000	
EDITOR_PASSWORD	change-me	Shared editor password until real accounts exist (Postgres "not started" per CLAUDE.md).
SESSION_SECRET	change-me-too	Long random strings in real environments — these sign session + embed tokens.
EMBED_TOKEN_SECRET	change-me-as-well	
DATABASE_URL	postgresql://postgres.<project-ref>:<password>@aws-0-ap-south-1.pooler.supabase.com:5432/postgres	Studio accounts in Postgres (src/usersStore.js). Unset: JSON files in DATA_DIR/users. Supabase: Project → Connect → "Session pooler" string (IPv4), with your DB password. Tables go in the `app` schema (not exposed by Supabase's Data API), created on start. Copy existing accounts in once with: <code>npm run db:import-users</code>
DATABASE_CA_FILE	./prod-ca-2021.crt	The provider's CA certificate, so TLS is verified (Supabase: Database → SSL configuration).
ASSET_DRIVER	local	Where uploaded splats are kept (src/storage.js). local this PC's disk (ASSET_DIR). Only this PC can show them. s3 one cloud bucket (Cloudflare R2, Backblaze B2, AWS S3, ...) that every PC and the hosted API share, so a splat uploaded anywhere shows everywhere. Copy existing uploads up with: <code>npm run assets:push</code>
ASSET_DIR	./src/data/assets	
S3_ENDPOINT		For ASSET_DRIVER=s3. Cloudflare R2: dashboard → R2 → create a bucket, then "Manage R2 API tokens" → Object Read & Write. The endpoint is <code>https://<account-id>.r2.cloudflarestorage.com</code> and the region is auto.
S3_BUCKET		
S3_REGION	auto	
S3_ACCESS_KEY_ID		
S3_SECRET_ACCESS_KEY		
S3_PREFIX	assets/	Optional: keys go under this folder in the bucket (default assets/).
SMS_TOKEN		Texts to guests (booking confirmations, waitlist news), besides email. Each is off until its keys are set; set one or both. SMS, through Sparrow SMS (sparrowsms.com): the token and sender identity from its dashboard.
SMS_FROM		
WHATSAPP_TOKEN		WhatsApp, through Meta's WhatsApp Cloud API (developers.facebook.com → WhatsApp): a permanent token, the sending number's id, and an approved message template with one <code>{{1}}</code> in its body. Meta lets a business message a guest first only through a template; without one, only guests who wrote within 24 h get it.
WHATSAPP_PHONE_ID		
WHATSAPP_TEMPLATE		
WHATSAPP_LANG	en	
PHONE_COUNTRY	977	The country code for phone numbers guests type without one.

Setting	Example	What it does
ANTHROPIC_API_KEY		The AI concierge in the tour: visitors ask questions and get answers from the project's published information (brand info, website, menu, rooms, spaces). Off until a key is set. From console.anthropic.com.
CONCIERGE_MODEL	claude-opus-5-5	The model: the most capable by default; claude-haiku-4-5-20251001 costs less.

Site settings

Setting	What it does
NEXT_PUBLIC_API_URL	Where the site finds the API. In production, the public domain (nginx sends <code>/api</code> to the API). <code>npm run dev</code> sets it.
NEXT_PUBLIC_SITE_URL	The public address, for canonical links, <code>sitemap.xml</code> , <code>robots.txt</code> and <code>llms.txt</code> . Falls back to the API URL in production.
NEXT_DIST_DIR	Build folder, so a second dev server or a test build doesn't collide with <code>.next</code> .

15 Deploying

One Linux VPS runs both programs under pm2, behind nginx with HTTPS. Scans live in a Cloudflare R2 bucket (`ASSET_DRIVER=s3`), so the server's disk holds only JSON in a `DATA_DIR` outside the checkout. The full steps are in `deploy/README.md`.

- Pushes to `main` that pass CI deploy themselves over SSH once the repository has the `DEPLOY_*` secrets; until then that job is skipped.
- Database migrations run when the API starts. There is nothing to run by hand.
- Put Cloudflare in front and add a cache rule for `/api/assets/*`: scan files are sent with a one-year immutable cache header.
- Back up `DATA_DIR` daily (and the database, if used). Scans are already in the bucket.

16 Recipes for common changes

Add an API endpoint

Add it to the right file in `server/src/routes/`, guard it (`requireEditorSession`, then `sceneGuard` or a project check), keep data access in a store module, write a comment above it (it becomes its description in this guide), and test it in `server/test/api.test.js`, refusals included.

Add a field to the scene document

Default it when absent, in `sceneDoc.ts` on load or in `migrate.js`. Sanitise any number that reaches the camera. Keep SDK concepts out of the schema. If a visitor needs it, make sure it survives publishing.

Add a studio edit

Add a store to `sceneDoc.ts` (load it in `loadSceneDoc`, write it in `sceneDocFor`), an entry in the history snapshot, and the UI in `EditorShell.tsx`.

Add a database table

Add `server/db/00N_name.sql` (tables in the `app` schema, row-level security on), and keep the store's exports the same as its file version so routes don't change.

Add a marketing page

Add `src/app/(site)/name/page.tsx` with a `canonical`, render it inside `SitePage`, and add its path to `PAGES` in `src/app/sitemap.xml/route.ts`.

Change a tour string

Wrap it in `t('...')` and add Nepali and Chinese to `i18n.ts`.

Before you finish

```
npx tsc --noEmit && npm test && npm run lint
npm run docs:dev-guide          # if routes, modules or settings changed
```

Then update `CLAUDE.md`, section 4, in the same change: a feature added, fixed or broken lands under Working, Needs fixing or Next up.

17 Known gaps

From `CLAUDE.md`, "Needs fixing", when this guide was built:

- **GitHub Pages must be switched off** (Settings → Pages → Source: None). It publishes the repo as a static page, never the app. The static-export code was removed on 2026-09-29.
- **Repo weight:** `.git` is ~760 MB (a 100 MB mp4 and an FBX were committed, now moved to `media-src/`). They remain in history, so shrinking it needs a history rewrite (`git filter-repo`), which is a team decision.
- **The API proxies every scan byte** from R2 through the VPS. Fine to start. Put Cloudflare in front with a cache rule for `/api/assets/*` before traffic grows.
- **Collision boxes:** no click-to-select in the 3D view (select from the tree), yaw-only rotation (no ramps), a box doesn't follow the model if the model is moved later (same as hotspots).
- Studio tab converts FBX/OBJ on the main thread, so a huge model freezes the tab during upload.
- Unreferenced uploaded files (removed audio, deleted spaces' old assets) are never garbage-collected.
- **Only accounts are in Postgres.** Projects, spaces, sites, bookings, leads, activity and stats are still JSON files in `DATA_DIR`. Move them one store at a time, behind the same exports. The API tests never use `server/.env`'s database (they set `DATABASE_URL` to `API_TEST_DATABASE_URL` or empty), and CI doesn't run them against Postgres yet.
- **Forgot password sends no real email yet:** `RESEND_API_KEY` (and a verified `LEADS_FROM` domain) aren't set on this PC.
- **Policy pages name Cloudflare storage and HTTPS**, which are only true once hosting is live (scans are on this PC's disk today).
- **Events:** no waitlist for a taken hall (rooms and tables have one), no calendar grid (a plain date field, with taken parts of the day struck out once a hall and day are picked), and no preview of real availability in the studio's website preview.
- **Room hotspot Book now checks no availability:** two guests can ask for the same room on the same night, and the team sorts it out when confirming. Undecided on purpose: a room hotspot may stand for one room or a room type with several. Halls and tables do check (2026-10-04), and reopening a declined request re-checks them. No waitlist for any of them.
- **Scan files aren't in git** (`server/src/data/assets/` is ignored). A new checkout or server has the scene JSON but no scans, and those tours show the start screen with no 3D. The studio's publish panel now warns (`routes/scenes.js missingFiles`). On 2026-10-04 eight were copied over from the older checkout; three exist nowhere on this PC and need uploading again: `computer-lab2`, `demo-project-map-plan`, `space-3d-model1`. Long-term fix: `ASSET_DRIVER=s3` (R2), so every PC and server reads the same files.
- Lint: 6 warnings (5 deliberate `<img>`, and `App.tsx` `onState` deps, checked: camera is stable and every `mgr` value is listed).